

Auto Dealer API

Backend completo para plataforma de compra e venda de veículos

Visão geral de funcionalidades, motivos de criação e uso de cada controle

Stack: Node.js · TypeScript · NestJS · PostgreSQL · Prisma ORM · Redis · Socket.IO/SSE · Docker

Repositório: github.com/knopfgit/site-coser (branch `main`)

Data do documento: 07/06/2026

Autor técnico: Backend gerado e documentado via Claude (Anthropic) — pronto para integração com o frontend

1. Objetivo deste documento

Este documento serve como **handoff técnico para o time de frontend** (e para uso com assistentes como Claude Code / ChatGPT durante a construção da interface). Ele explica **o que foi construído, como foi construído, por que cada peça existe e como ela deve ser usada** a partir do frontend. A documentação operacional detalhada (endpoints, payloads, eventos, exemplos) está em `docs/*.md` e na coleção `docs/postman_collection.json` dentro do repositório — este PDF é o "mapa" que conecta tudo isso.

2. Arquitetura geral — o que foi feito e por quê

O backend foi construído em **NestJS** (framework Nodejs modular sobre Express, com injeção de dependência e forte tipagem TypeScript), seguindo arquitetura em camadas por módulo de domínio: `Controller` → `Service` → `Prisma (ORM)` → `PostgreSQL`, com infraestrutura transversal (auth, cache, storage, e-mail, realtime, auditoria) compartilhada entre módulos.

Por que NestJS + Prisma + PostgreSQL? Era o stack exigido pela especificação do projeto. NestJS organiza o código em módulos coesos (cada domínio — veículos, vendas, clientes — vira um módulo isolado, testável e substituível), o que facilita tanto a manutenção quanto a divisão de trabalho entre back e frontend. Prisma fornece tipagem ponta-a-ponta do banco (o TypeScript "enxerga" as tabelas), migrações versionadas e proteção contra SQL injection. PostgreSQL é robusto para dados relacionais com integridade referencial — essencial num domínio com dezenas de entidades interligadas (veículo → estoque → venda → comissão → DRE).

Como o frontend usa isso na prática: o frontend nunca acessa o banco — fala exclusivamente HTTP/WebSocket com a API REST em `/api` (ou o gateway de realtime). Toda a estrutura de dados retornada já vem tipada e estável, documentada em `docs/DATABASE_MODEL.md` e `docs/API_ENDPOINTS.md`.

2.1 Estrutura de pastas

```
src/
├─ main.ts, app.module.ts      # bootstrap e wiring global
├─ config/                     # leitura tipada do .env
├─ common/                     # guards, decorators, filtros, interceptors, validadores BR
├─ prisma/ redis/ storage/ mail/ audit/ realtime/ # infraestrutura compartilhada
├─ modules/
│   ├── auth, users, employees, customers
│   ├── vehicle-specs, vehicles (+ stock.service)
│   ├── parts (+ suppliers), maintenance
│   ├── documents
│   ├── commercial (acquisitions, reservations, sales), financial (DRE), commissions
│   ├── leads, store, privacy, notifications, dashboard, public, reports
│   └─ jobs
├─ prisma/                     → schema.prisma, migrations/, seed.ts
├─ docs/                       → 10 manuais .md + coleção Postman
└─ test/                       → testes e2e
```

3. Padrão de resposta da API (envelope) — motivo e uso

Motivo: sem um formato único, cada endpoint retornaria estruturas diferentes, forçando o frontend a tratar casos especiais. Um envelope padronizado permite que o frontend escreva **um único interceptor HTTP** (Axios/Fetch) capaz de processar qualquer resposta, sucesso ou erro, de forma previsível — reduzindo bugs de integração e acelerando o desenvolvimento.

Tipo	Formato
Sucesso (item único)	<code>{ "success": true, "data": {...}, "meta": { "timestamp": "..."} }</code>
Sucesso (lista paginada)	<code>{ "success": true, "data": [...], "meta": { "page", "limit", "total", "totalPages", "timestamp"} }</code>
Erro	<code>{ "success": false, "error": { "code", "message", "details"}, "meta": { "timestamp", "path"} }</code>

Como foi implementado: um `ResponseInterceptor` global envolve toda resposta de sucesso (detectando automaticamente quando o retorno é um `PaginatedResult` para preencher `meta`); um `AllExceptionsFilter` global captura qualquer exceção (validação, regra de negócio via `AppException`, ou erro interno) e a serializa no mesmo formato de erro, sempre com um `error.code` estável (ex.: `VEHICLE_NOT_FOUND`, `VALIDATION_ERROR`) — o frontend deve usar esse `code` para lógica condicional (ex.: redirecionar em `UNAUTHORIZED`) e a `message` (em pt-BR) para exibir ao usuário.

4. Autenticação, sessão e RBAC — motivo e uso

Motivo: a plataforma tem três perfis com visões e permissões muito diferentes (ADMIN vê tudo da loja; SELLER só seus próprios clientes/comissões; CUSTOMER só seus próprios dados). Um esquema de autenticação robusto evita vazamento de dados sensíveis (comissão de outro vendedor, faturamento da loja, dados pessoais de outro cliente) e cumpre LGPD.

Como funciona / como o frontend integra:

- **Login:** `POST /auth/login` retorna `accessToken` (curta duração, ~15 min) + `refreshToken` (revogável, armazenado no banco como hash SHA-256 — nunca em texto puro). O frontend envia o access token em `Authorization: Bearer <token>` em toda chamada autenticada.
- **Renovação automática:** ao receber `401 UNAUTHORIZED`, o frontend chama `POST /auth/refresh`; o backend faz *rotação* do refresh token (o antigo é invalidado e um novo é emitido) — isso limita o dano caso um token vaz.
- **RBAC:** cada endpoint é protegido por `@Roles('ADMIN' | 'SELLER' | 'CUSTOMER')` + um guard global (`JwtAuthGuard` → `RolesGuard`); rotas abertas usam `@Public()`. O backend nunca confia no "papel" enviado pelo cliente — ele é resolvido a partir do token assinado no servidor.
- **Bloqueio de login (lockout):** após várias tentativas inválidas, a conta é bloqueada temporariamente (`ACCOUNT_LOCKED`) — o frontend deve exibir mensagem amigável e sugerir "esqueci minha senha".
- **Sessão no frontend:** guarde o access token em memória (estado da aplicação) e o refresh token com cuidado (idealmente um BFF com cookie httpOnly); nunca exponha tokens em logs ou URLs.

Papel	O que enxerga / pode fazer
ADMIN	Acesso total: cadastro de loja, usuários/funcionários, todos os veículos, estoque, peças, manutenção, documentos, vendas, comissões de todos os vendedores, DRE consolidada e por veículo, dashboards globais, relatórios, auditoria, configurações.
SELLER	Veículos e clientes que atende, suas próprias reservas/vendas/comissões e seu próprio dashboard — nunca vê comissão de outro vendedor nem faturamento consolidado da loja.
CUSTOMER	Autoatendimento: seu cadastro/endereço, suas compras, documentos relacionados a si, favoritos, histórico de visualização, preferências de marketing e solicitações LGPD — nunca dados de outro cliente.

5. Módulos de domínio — o que cada um faz, por que existe e como o frontend o usa

5.1 Auth · Users · Employees · Customers

O que é: autenticação (login/registro/refresh/recuperação de senha), gestão de papéis e permissões (RBAC), CRUD de funcionários (vendedores/equipe interna) e de clientes (com endereços).

Por quê: é a base de identidade de todo o sistema — sem ela nenhuma regra de negócio (quem pode ver o quê, quem é "o vendedor" de uma venda, quem é "o cliente" de um documento) seria possível.

Uso no frontend: telas de login/cadastro/recuperação de senha; área "minha conta" do cliente (`/customers/me`); telas administrativas de gestão de equipe e clientes (apenas ADMIN).

5.2 VehicleSpecs · Vehicles · Stock

O que é: ficha técnica de veículos com **preenchimento automático desacoplado** (padrão `VehicleSpecsProvider` — uma interface que pode ser implementada por qualquer fonte de dados externa, hoje atendida por um provedor "mock" com catálogo realista de marcas/modelos brasileiros, cacheado em Redis); cadastro de veículos com **separação rígida entre campos públicos e internos**; e máquina de estados de estoque (`DRAFT` → `AVAILABLE` → `RESERVED` → `SOLD` → `ARCHIVED` , entre outros) com transições validadas.

Por quê: (1) digitar a ficha técnica de cada veículo manualmente é lento e sujeito a erro — o preenchimento automático acelera o cadastro; (2) o catálogo público **não pode** revelar preço de compra, margem, notas internas, placa/chassi/Renavam (dados sensíveis e estratégicos) — por isso existe um *serializer* dedicado que filtra esses campos antes de qualquer resposta pública; (3) sem uma máquina de estados, seria possível (por erro de UI ou requisição manipulada) "vender" um carro arquivado ou reservar um já vendido — a validação de transições impede isso na origem.

Uso no frontend:

- Catálogo público: `GET /public/vehicles` , `/public/vehicles/:slug` , `/public/vehicles/featured` , `/public/filters` — sempre retornam o objeto *público* (sem campos internos).
- Área interna (ADMIN/SELLER): `GET/POST/PATCH /vehicles` retornam o objeto completo; `POST /vehicles/:id/apply-specs` aciona o preenchimento automático; `POST /vehicles/:id/status` muda o estágio no funil de estoque (o backend recusa transições inválidas com `VEHICLE_INVALID_STATUS_TRANSITION`).

5.3 Parts (Peças) · Suppliers (Fornecedores) · Maintenance (Manutenção)

O que é: controle de estoque de peças (entradas/saídas/ajustes com custo médio), cadastro de fornecedores e ordens de manutenção/revisão de veículos que consomem peças e geram custo de mão de obra.

Por quê: o custo de "deixar um carro pronto para venda" é parte central da lucratividade (DRE). Sem rastrear consumo de peças e mão de obra por veículo, seria impossível calcular o lucro real de cada venda. O alerta de estoque mínimo evita que a oficina pare por falta de peça.

Uso no frontend: telas internas de gestão de estoque de peças (`/parts`), fornecedores (`/suppliers`) e ordens de manutenção (`/maintenance`) — ao concluir uma manutenção, o backend lança automaticamente os custos na DRE do veículo e dispara eventos em tempo real (`maintenance.completed` , `part.low_stock`).

5.4 Documents (Documentos)

O que é: tipos de documento e checklists **configuráveis** (o admin define quais documentos são obrigatórios para cada fluxo), upload com versionamento, validação e **download restrito por permissão**.

Por quê: processos de venda de veículo exigem documentação variável (CRLV, laudo cautelar, contrato, comprovantes) que muda conforme o tipo de negociação — torná-la configurável evita alterar código sempre que o processo da loja mudar. O download restrito impede que um cliente baixe documento de outro, ou que dados internos vazem.

Uso no frontend: upload via `multipart/form-data` (`POST /documents/upload`); exibição de checklist de pendências por veículo/venda (`GET /documents/checklist-status/...`); download sempre via endpoint autenticado que valida se o usuário tem

direito ao arquivo.

5.5 Commercial (Aquisição · Reserva · Venda) · Financial/DRE · Commissions

O que é: o núcleo comercial — registrar a *aquisição* de um veículo (entrada no estoque), *reservar* para um cliente, *concluir a venda*; em paralelo, o motor de **DRE (Demonstrativo de Resultado)** por veículo e consolidado, e o motor de **comissões** com regras flexíveis.

Por quê: esse é o "coração financeiro" do negócio — é aqui que a loja descobre se está tendo lucro. Concentrar essas regras em serviços dedicados (em vez de espalhá-las pelo frontend) garante que o cálculo seja *sempre* o mesmo, auditável e não manipulável pelo cliente.

Como o cálculo funciona (motivo de cada peça):

- **DRE** agrega lançamentos financeiros (`FinancialEntry` : custo de aquisição, peças, mão de obra, despesas) e o total de comissões geradas para aquele veículo, calculando `totalInvested`, `totalExpenses`, `totalRevenue`, `grossProfit`, `netProfit`, `margin`, `daysInStock`, `costPerDay`. Comissões nunca são contadas em duplicidade: ficam exclusivamente na tabela `Commission` e são somadas separadamente no resultado líquido — o serviço de DRE explicitamente ignora qualquer `FinancialEntry` de categoria "Comissão" ao somar despesas.
- **Comissões** suportam 4 modelos configuráveis — `PERCENT_SALE` (percentual sobre o valor da venda), `PERCENT_PROFIT` (percentual sobre o lucro), `FIXED` (valor fixo) e `PROGRESSIVE` (faixas/tiers escalonados) — porque lojas reais usam políticas de remuneração diferentes e essa política pode mudar com o tempo sem exigir nova versão do backend.
- **Fluxo de venda** (`sales.service.ts`) ao concluir uma venda (`PATCH /sales/:id/status → COMPLETED`) automaticamente: muda o status do veículo para `SOLD`, gera o lançamento de receita na DRE, calcula e cria a(s) comissão(ões) do(s) vendedor(es) envolvidos, e dispara eventos em tempo real (`sale.completed`, `commission.generated`, `dashboard.updated`).

Uso no frontend: funil comercial (`/acquisitions`, `/reservations`, `/sales`); telas de DRE (`/dre/vehicle/:id` para análise individual, `/dre/consolidated` para visão gerencial — apenas ADMIN); extrato de comissões pessoal do vendedor (`/commissions/me`, sempre filtrado pelo próprio `employeeId`).

5.6 Leads · WhatsApp ("Falar com especialista")

O que é: captação de leads (potenciais compradores) com distribuição automática entre vendedores (*round-robin* ou *least-busy*) e geração do link de WhatsApp pré-preenchido.

Por quê: o WhatsApp é o canal de conversão dominante no mercado automotivo brasileiro. Automatizar a distribuição evita que um único vendedor fique sobrecarregado (ou que leads "se percam" sem dono) e dá rastreabilidade ao funil de vendas (de onde veio o lead, quem atendeu, taxa de conversão).

Uso no frontend: botão "Falar com especialista" na página do veículo chama `POST /public/leads/specialist-contact` (endpoint público, sem login) e o frontend abre a `whatsappUrl` retornada (`window.open(url, '_blank')`) — formato `https://wa.me/<telefone>?text=...` já com mensagem pré-preenchida contendo dados do veículo.

5.7 Store (Loja/Mapa)

O que é: dados de localização da loja (endereço, coordenadas, horário, redes sociais) com geração de URLs do Google Maps/"como chegar".

Por quê: centralizar essa informação no backend permite trocá-la (endereço, telefone, horário) sem precisar de novo deploy do frontend — útil para lojas físicas que mudam de endereço ou ajustam horário sazonalmente.

Uso no frontend: `GET /public/store/location` alimenta o mapa (coordenadas) e o botão "Como chegar" (`directionsUrl`).

5.8 Privacy / LGPD (Consentimento, Rastreamento, Favoritos)

O que é: registro de consentimento de cookies por categoria (essencial, analytics, marketing, localização), rastreamento de visualizações e localização *somente com consentimento*, favoritos, preferências de marketing, e fluxo de solicitações LGPD (exportação e exclusão de dados).

Por quê: a Lei Geral de Proteção de Dados (LGPD) exige consentimento explícito antes de coletar dados pessoais/comportamentais, e garante ao titular o direito de exportar ou apagar seus dados. Construir isso desde o início evita retrabalho jurídico e técnico mais tarde, e protege a empresa de sanções.

Como foi implementado (detalhe técnico relevante para o frontend): o IP do visitante **nunca** é armazenado em texto puro — é transformado em hash SHA-256 antes de qualquer persistência (`ClientInfoParam`). O envio de localização (`POST /tracking/location`) só é aceito se já existir consentimento de categoria `LOCATION` registrado — o frontend deve sempre checar/solicitar o consentimento *antes* de chamar esse endpoint, e exibir o banner de cookies usando `POST /consents` com a lista de categorias.

5.9 Notifications (Notificações)

O que é: notificações in-app (lidas/não lidas, por usuário) e fila de e-mails transacionais (boas-vindas, redefinição de senha, confirmação de venda, alertas de estoque etc.).

Por quê: manter usuários informados de eventos relevantes (nova reserva, venda concluída, comissão aprovada, documento pendente) aumenta engajamento e reduz follow-up manual. A fila assíncrona evita que o envio de e-mail trave a resposta da API.

Uso no frontend: sino de notificações consultando `GET /notifications` + `PATCH /notifications/:id/read` ; pode (e deve) ser combinado com o evento realtime `notification.created` para atualização instantânea sem precisar recarregar a página.

5.10 Dashboard · Realtime (WebSocket/SSE)

O que é: indicadores gerenciais por papel (admin vê a loja inteira; vendedor vê seu próprio desempenho) atualizados **em tempo real** via WebSocket (Socket.IO) ou Server-Sent Events, com fan-out via Redis pub/sub (permitindo múltiplas instâncias da API).

Por quê: dashboards "estáticos" (que exigem F5) dão sensação de atraso e geram mais chamadas HTTP desnecessárias (polling). Tempo real torna a operação mais ágil — ex.: o vendedor vê instantaneamente quando recebe um novo lead, o admin vê o funil de vendas se mover ao vivo.

Uso no frontend:

```
import { io } from 'socket.io-client';
const socket = io('http://localhost:3000/realtime', { auth: { token: accessToken } });
socket.on('dashboard.updated', () => refetchDashboard());
socket.on('lead.assigned', (msg) => toast('Novo lead!', msg.data));
```

O backend distribui eventos por "salas" (role:ADMIN, role:SELLER, user:<id>, seller:<id>), de forma que cada cliente só recebe o que tem permissão de ver. A lista completa dos 17 eventos está em `docs/REALTIME_EVENTS.md` (ex.: `vehicle.status_changed`, `sale.completed`, `commission.generated`, `part.low_stock`, `lead.assigned`, `dashboard.updated`, `notification.created`).

5.11 Public (Catálogo público) · Reports · Audit · Jobs

Módulo	O que faz	Por que existe / como usar
Public	Conjunto de endpoints sem autenticação que expõem <i>apenas</i> o que é seguro mostrar publicamente (catálogo, filtros, localização, contato).	Permite construir páginas públicas (SEO-friendly, sem exigir login) com a garantia de que nenhum campo interno será exposto — toda a filtragem acontece no backend, nunca depende do frontend "esconder" campos.
Reports	18 tipos de relatórios (estoque, vendas, comissões, DRE, manutenção, leads etc.) exportáveis em CSV ou JSON.	Gestores precisam analisar dados fora da plataforma (Excel, BI). Exportação server-side garante que os dados batem com o que está no sistema e respeita as mesmas regras de permissão (um vendedor não consegue exportar relatório de outro).
Audit	Registro de quem fez o quê, quando e por quê (criação/edição/exclusão de registros sensíveis).	Rastreabilidade e responsabilização — essencial em qualquer sistema que lida com dinheiro, documentos e dados pessoais; também é exigência indireta da LGPD (comprovar o que foi feito com os dados de um titular).

Jobs (cron)	8 tarefas agendadas: expiração de reservas, alerta de estoque baixo, recálculo de dashboards, processamento da fila de e-mails, limpeza de tokens expirados, anonimização de dados (LGPD), verificação de revisões/documentos a vencer etc.	Mantém o sistema "saudável" sem intervenção manual — ex.: uma reserva não paga expira sozinha e libera o veículo de volta ao estoque; dados de titulares que pediram exclusão são anonimizados automaticamente conforme a política definida.
------------------------	---	--

6. Controles de segurança — motivo de cada um e como ele afeta o frontend

Controle	Motivo da criação	Como afeta/é usado pelo frontend
Hash de senha (bcrypt)	Senhas nunca devem ser recuperáveis em texto puro — mesmo que o banco vaze, o atacante não obtém a senha original.	Transparente; o frontend só envia a senha em texto puro <i>uma vez</i> , via HTTPS, no login/cadastro.
JWT curto + refresh token rotativo e revogável	Reduz a janela de uso de um token roubado; permite "deslogar remotamente" revogando o refresh token no banco.	Implementar o interceptor de renovação automática (ver seção 4); tratar <code>INVALID_REFRESH_TOKEN</code> como "sessão expirada, faça login novamente".
Guards globais de autenticação e papel (RBAC)	Garantir, no servidor, que ninguém acesse dado que não deveria — o frontend pode (e deve) também esconder botões/menus por papel, mas a <i>autoridade</i> de fato é sempre do backend.	Tratar <code>FORBIDDEN</code> (403) como "ação não permitida para este perfil" e ajustar a UI condicionalmente ao <code>role</code> retornado em <code>/auth/me</code> .
Rate limiting (Throttler)	Protege contra abuso/força bruta e picos que poderiam derrubar o serviço.	Tratar <code>429 / RATE_LIMITED</code> com mensagem de "muitas tentativas, aguarde alguns instantes".
Helmet + CORS configurado	Cabeçalhos HTTP de segurança (XSS, sniffing, clickjacking) e controle de quais origens podem chamar a API.	O frontend deve rodar a partir de uma origem liberada em <code>CORS_ORIGIN</code> (configurável via <code>.env</code>).
Validação de DTOs (class-validator) + <code>whitelist: true</code>	Garante que apenas os campos esperados sejam aceitos — previne <i>mass assignment</i> (cliente tentando setar campos como <code>role</code> ou <code>customerId</code> de outra pessoa). Durante a revisão final, encontramos e corrigimos exatamente esse risco no endpoint <code>PUT /public/marketing/preferences</code> .	Erros de validação chegam como <code>VALIDATION_ERROR</code> com lista de campos em <code>error.details</code> — o frontend deve mapear isso para mensagens de campo no formulário.
Límites e validação de upload (tamanho/MIME/nome seguro)	Evita ataques via upload (arquivos enormes, executáveis disfarçados, nomes maliciosos que sobrescrevem outros arquivos).	Tratar <code>UPLOAD_TOO_LARGE</code> e <code>UPLOAD_INVALID_TYPE</code> com orientação clara (tamanho/Tipo aceitos) antes mesmo de enviar, validando no cliente como primeira camada.
Download de documentos restrito por permissão	Impede que, sabendo a URL/ID, alguém baixe um documento de outro cliente ou um arquivo interno.	Sempre buscar o link de download via endpoint autenticado (nunca montar URLs "na mão").
Bloqueio de login (lockout) por tentativas	Mitiga ataques de força bruta a senhas.	Exibir mensagem amigável e oferecer fluxo de recuperação de senha quando receber <code>ACCOUNT_LOCKED</code> .
Auditoria (audit trail)	Rastreabilidade de ações sensíveis para investigação e conformidade.	Tela de auditoria (apenas ADMIN) consumindo <code>/audit</code> .
Hash de IP + consentimento prévio (LGPD)	Minimizar dados pessoais armazenados e cumprir a lei.	Implementar banner de cookies <i>antes</i> de qualquer rastreamento; nunca chamar endpoints de tracking sem que o usuário tenha consentido.

7. Infraestrutura compartilhada — o que existe "por baixo dos panos" e por quê

Peça	Função	Motivo
Prisma + PostgreSQL	ORM tipado e banco relacional.	Integridade referencial entre ~40 entidades interligadas; migrações versionadas garantem que todo ambiente (dev, homologação, produção) tenha o mesmo esquema.

Redis	Cache (ex.: ficha técnica de veículos) e backbone de pub/sub para eventos realtime entre instâncias.	Reduz latência/repetição de chamadas a provedores externos e permite escalar a API horizontalmente sem perder eventos em tempo real.
Storage abstrato (local → S3-ready)	Interface única <code>StorageProvider</code> com implementação local funcional hoje e <i>scaffold</i> para Amazon S3.	Permite trocar "onde os arquivos ficam" (disco local em desenvolvimento, S3/bucket em produção) sem alterar nenhuma linha dos módulos de negócio — só trocar a configuração.
Mailer (Nodemailer)	Envio de e-mails via fila, com templates centralizados e driver configurável (console em dev / SMTP em produção; testável via MailHog no Docker Compose).	Centraliza textos/identidade visual dos e-mails e evita travar requisições aguardando o envio.
VehicleSpecsProvider	Interface desacoplada para "de onde vêm" os dados técnicos do veículo.	Hoje atendida por um provedor mock (catálogo offline com dados reais de marcas/modelos), com fallback gracioso (se falhar, retorna vazio em vez de quebrar o cadastro) — amanhã pode ser trocado por uma API paga (ex.: FIPE/Vehicle data) <i>sem mudar o restante do sistema</i> .

8. Qualidade — testes, lint e build

- **Build TypeScript:** compila sem erros (`npm run build`).
- **Lint (ESLint + Prettier):** 0 erros, 0 warnings.
- **Testes unitários:** 7 suítes / 25 testes, **todos passando**, cobrindo validadores brasileiros (CPF/CNPJ/CEP), exportação CSV, serialização pública de veículo (garantindo que campos internos nunca vazam), máquina de estados de estoque, distribuição de leads, cálculo de comissões e o provedor mock de fichas técnicas.
- **Testes e2e:** escritos e prontos (`test/app.e2e-spec.ts`), a serem executados com a infraestrutura (Docker) ativa.

9. Documentação entregue (no repositório)

Arquivo	Conteúdo
<code>docs/PROJECT_SETUP.md</code>	Como instalar/rodar o projeto.
<code>docs/ENVIRONMENT_VARIABLES.md</code>	Todas as variáveis de ambiente explicadas.
<code>docs/AUTHENTICATION.md</code>	Fluxo completo de autenticação/refresh.
<code>docs/ROLES_AND_PERMISSIONS.md</code>	Matriz de permissões por papel e endpoint.
<code>docs/REALTIME_EVENTS.md</code>	Os 17 eventos em tempo real, payloads e quem os recebe.
<code>docs/API_ENDPOINTS.md</code>	Lista completa de endpoints por módulo.
<code>docs/DATABASE_MODEL.md</code>	Modelo de dados (entidades, relações, enums).
<code>docs/BUSINESS_RULES.md</code>	Regras de negócio (transições de estoque, cálculo de DRE/comissões, fluxos comerciais).
<code>docs/EXAMPLE_REQUESTS.md</code>	Exemplos reais de requisições e respostas.
<code>docs/FRONTEND_INTEGRATION.md</code>	Guia dedicado ao time de frontend — autenticação, upload, paginação, filtros, erros, eventos realtime e fluxos completos com exemplos Axios/Fetch.
<code>docs/postman_collection.json</code>	Coleção Postman pronta, com captura automática de token ao logar.

10. Como executar o backend (para desenvolvimento integrado com o frontend)

```
git clone https://github.com/knopfgit/site-coser.git
cd site-coser
cp .env.example .env
docker compose up -d      # PostgreSQL + Redis + MailHog
npm install
npx prisma migrate dev
npx prisma db seed
npm run start:dev          # API em http://localhost:3000/api
```

Recurso	URL
API	<code>http://localhost:3000/api</code>
Swagger (documentação interativa/viva)	<code>http://localhost:3000/docs</code>
WebSocket realtime	<code>ws://localhost:3000/realtime</code>
SSE realtime	<code>GET /api/realtime/stream</code>
MailHog (e-mails de teste)	<code>http://localhost:8025</code>

Credenciais de desenvolvimento (seed)

Papel	E-mail	Senha
ADMIN	admin@autodealer.local	Admin@123
SELLER	carlos@autodealer.local / ana@autodealer.local	Seller@123
CUSTOMER	maria@cliente.com / joao@cliente.com	Customer@123

11. Pontos preparados para integração futura

- **Armazenamento em nuvem:** trocar `STORAGE_DRIVER=local` por `s3` e instalar `@aws-sdk/client-s3` — nenhuma mudança de código de negócio é necessária.
- **Ficha técnica via API paga (ex.: FIPE):** implementar a interface `VehicleSpecsProvider` e registrá-la no módulo — o resto do sistema continua funcionando sem alterações.
- **Geração de PDF de relatórios:** os serviços de relatório já retornam os dados prontos ("datasets"); falta apenas plugar uma camada de renderização (ex.: Puppeteer/PDFKit).
- **Escalabilidade horizontal:** eventos em tempo real já trafegam via Redis pub/sub — basta subir mais instâncias da API atrás de um load balancer.

12. Limitações conhecidas

- O preenchimento automático de ficha técnica usa um **provedor mock offline** (catálogo com dados ilustrativos de marcas/modelos reais) — não depende de nenhuma API paga para funcionar, mas pode ser substituído por uma fonte oficial quando desejado.
- O provedor de armazenamento em S3 está implementado como *scaffold* (estrutura pronta, porém requer a instalação da SDK da AWS e configuração de credenciais para uso real).
- Testes e2e completos exigem a infraestrutura Docker ativa (Postgres/Redis) — não puderam ser executados no ambiente isolado de geração do código, mas estão escritos e prontos para rodar localmente.

Documento gerado como parte do handoff técnico do backend "Auto Dealer API" para o time de frontend. Para detalhes operacionais (payloads exatos, exemplos de requisição/resposta, lista completa de endpoints e eventos), consulte sempre os arquivos em `docs/` no repositório — eles são a fonte de verdade viva, e o Swagger (`/docs`) reflete o estado real da API a cada execução.